How Ije was built

The decisions behind the work, explained

A plain-language guide to what changed in Ije on 15 October 2026, and why each important coding decision was made the way it was.

Ije means journey in Igbo.

Pull requests #1 to #3 · ADRs 006 and 007 · Written with Claude Code

Ije

How to read this

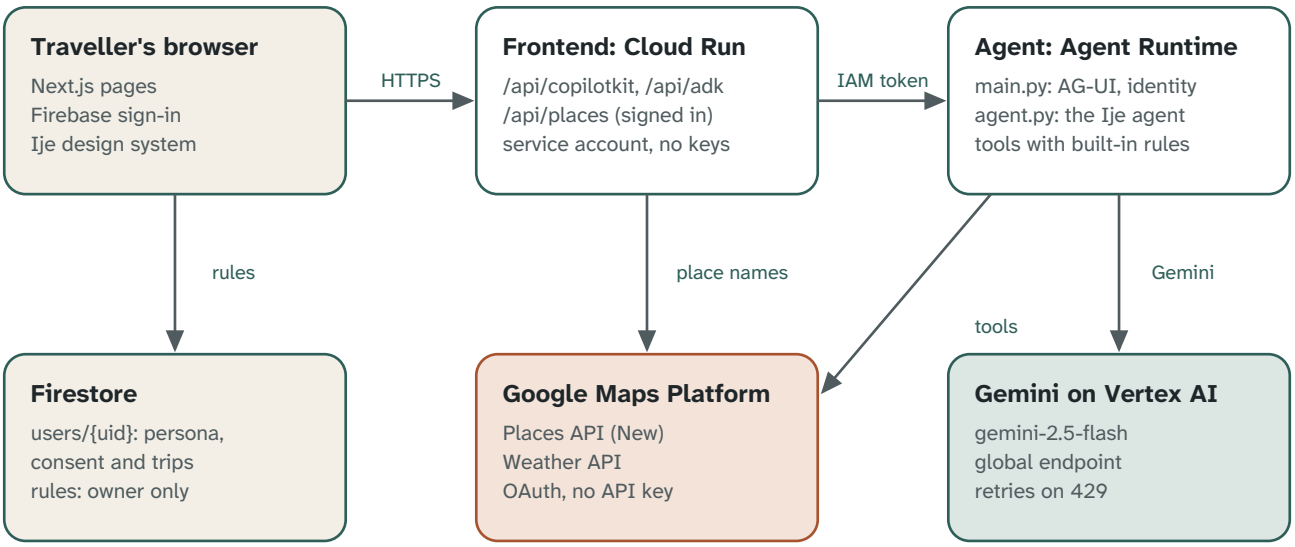
This guide explains the reasoning behind the work, not just the result. Each important choice is written as a decision card: the question I faced, what I chose, why, and what I deliberately did not do. You do not need to read code to follow it; where code matters, the file is named so you can look.

Part	What it covers
1. The big picture	How the pieces fit, and the six rules behind every decision
2. Signing in and guests	Where people land after sign-in, and why guests are never pushed
3. Protecting health data	Consent, deletion and the database rules that enforce them
4. The planning experience	Light theme, the trip form, the itinerary canvas and the chat
5. Making the agent great	Real places and weather, and rules built into the tools
6. Proving it works	The evals: what they measure and what they caught
7. Shipping it	Deploying the agent and the website safely
8. Honest notes	What I chose not to do, mistakes I made, what is still open
Glossary	Plain meanings of the technical terms

PART 1

The big picture

Ije is a travel planner. A traveller signs in, tells Ije how they travel, describes a trip, and gets a day-by-day itinerary built by an AI agent from real places and real weather.



Every agent request carries the user's Firebase ID token; the agent checks it before anything else runs.

Three parts do the work. The **website** (Next.js on Cloud Run) is what the traveller sees. The **agent** (Google's Agent Development Kit on Agent Runtime) does the planning. **Firestore** keeps each person's profile and trips. The agent asks Google Maps Platform for places and weather, and Gemini for the thinking.

The six rules behind every decision

Rule	What it means in practice
Identity comes only from a verified token	The agent trusts nobody's word about who a user is. Every request carries a Firebase ID token, and the agent checks it with Google before doing anything.
No API keys anywhere	Keys leak. Every Google service is reached with short-lived tokens from a named identity instead (ADR 002).
No new third parties without an ADR	Before any outside service sees user data, the decision is written down and agreed. Google Maps Platform got ADR 006.
Important rules live in code, not only in the prompt	An instruction to an AI model is followed most of the time. A rule in code is followed every time. The evals showed the difference.
Store only what we are allowed to	Health data needs consent. Google's terms allow storing place IDs but not place names. The code is shaped around both.

Rule	What it means in practice
Measure, then believe	Agent quality is scored by evals before every deploy, including cases kept back so they cannot be gamed.

PART 2

Signing in and guests

The first job was a simple one: after signing in, send each person to the right place.

DECISION 1

One place decides where you land

THE QUESTION	After sign-in, how does the app know whether to show onboarding or your trips?
WHAT I CHOSE	The home page (**app/page.tsx**) reads your profile from Firestore. No saved preferences means onboarding; otherwise your trips. Other pages only check their own preconditions.
WHY	One place to decide means one place to fix. Basing it on stored data (not a flag in the browser) means it is right on any device, and for a guest who later creates an account.
WHAT I DID NOT DO	I did not add a separate "is new user" flag. The profile already answers the question, and a second source of truth can disagree with the first.

DECISION 2

Guests are never pushed to sign up

THE QUESTION	The old placeholder told guests to create an account or sign out. Should it?
WHAT I CHOSE	Guests use everything a full account can. Saving the account is offered once, quietly, in the account menu. Signing out as a guest asks for confirmation inside the menu, because it loses their trips.
WHY	Asking for commitment before someone has seen any value loses them. Firebase lets a guest become a full account and keep the same identity, so waiting costs nothing.
WHAT I DID NOT DO	I did not use the browser's built-in confirm popup. It looks foreign, cannot be styled, and is blocked in some embedded views.

PART 3

Protecting health data

Diet, allergies and access needs can reveal health or religion. UK law calls this special category data and treats it with extra care.

IN PLAIN WORDS

A note on a screen saying "we keep this" is not consent. Consent means a clear, separate, unticked choice that can be taken back. Before this work, Ije only showed a note.

DECISION 3

A real checkbox, unticked, before any health fields appear

THE QUESTION	How should Ije ask permission to keep diet, allergy and access needs?
WHAT I CHOSE	One unticked checkbox with plain wording. The health fields only appear once it is ticked, so nothing is collected before consent. It covers the traveller and their travel group.
WHY	Explicit consent is the most likely lawful condition for this data. Hiding the fields until consent means the app cannot collect them by accident.
WHAT I DID NOT DO	I did not style it as one of Ije's toggle chips. Consent is a yes or no, and a real checkbox is what screen readers and people expect for that.

DECISION 4

Consent has a version number

THE QUESTION	What happens if the consent wording changes later?
WHAT I CHOSE	The saved consent records the wording version it was given to. If the wording changes, the version changes, and people are asked again.
WHY	Consent only counts for what the person actually agreed to. A version makes that checkable.

DECISION 5

Withdrawing consent deletes the data everywhere, in one step

THE QUESTION	If someone unticks the box, what exactly happens?
WHAT I CHOSE	Saving clears the health fields from the profile and from the group needs on every trip, in one atomic batch: either all of it is deleted or none of it.
WHY	Half a deletion would leave health data behind, and the database rules would then refuse later saves to those trips. One batch avoids both.

DECISION 6

The database refuses health data without consent

THE QUESTION	The app already hides the fields. Why also enforce it in Firestore?
WHAT I CHOSE	The Firestore rules refuse any health field unless consent to the current wording is on record, and only ever let a person read and write their own data.
WHY	Anyone can call Firestore directly with the public web config. The rules are the only thing that cannot be bypassed from a browser. Belt and braces.

DECISION 7

Test the rules, and test the tests

THE QUESTION	How do we know the rules actually work?
WHAT I CHOSE	29 automated tests run against Google's Firestore emulator. They use the exact data shapes the app writes. Then I deliberately broke the rules three ways and confirmed the right tests failed.
WHY	Rules that are not tested are a guess. Testing hand-copied data can pass while the real app fails. Breaking the rules on purpose proves the tests would notice a real mistake.

DECISION 8

No privacy policy yet: a worksheet instead

THE QUESTION	Should Ije publish a privacy policy now?
WHAT I CHOSE	A governance worksheet (**docs/privacy-governance.md**) records what is collected, where it goes, and the decisions a policy depends on. The policy will be written from it.
WHY	You were right that it needed the governance phase first. A policy that describes undecided things is worse than none.

PART 4

The planning experience

You tried the app and found two problems: a large dark screen, and a dead end after onboarding.

DECISION 9

Light only, switched off at the source

THE QUESTION	The app followed the computer's dark mode into a near-black palette you did not like. How to stop it?
WHAT I CHOSE	A setting in the design tokens (**followSystem: false**). The generator simply stops writing the dark-mode rule. The dark colours stay on record for a possible opt-in later. I also replaced Next.js's default 404 page, which had its own dark mode.
WHY	Fixing it where the colours are generated means no override fighting the system. One switch, one place, easy to undo.

DECISION 10

New travellers go straight to planning

THE QUESTION	After onboarding, Ije showed "No trips yet" with nothing to do. What should happen?
WHAT I CHOSE	A first-time traveller goes straight from onboarding to the trip form. An empty trips list redirects there too. The list only appears once there is something in it.
WHY	Every screen should have a next step. An empty list is a dead end.

DECISION 11

Ask who is going before asking about money

THE QUESTION	The form asked for a "total budget for the group" before knowing it was a group.
WHAT I CHOSE	Who is going comes first. Then the budget appears, labelled for one person or for the group, and group needs only appear for group trips. Solo and couple fix the traveller count automatically.
WHY	This was your observation, and it was right: a question should make sense when it is asked. It also matches the rules in the data model, which require one traveller for solo and two for a couple.

DECISION 12

A trip is at most 14 days

THE QUESTION	Should the form accept any length of trip?
WHAT I CHOSE	Trips are capped at 14 days per plan, with a clear message.
WHY	Each day needs three picks, so a long trip makes one very large request to the AI, which is slower and more likely to fail. The cap is one number in the code and easy to change.

DECISION 13

One trip, one conversation

THE QUESTION	How should the agent keep each trip's chat separate?
WHAT I CHOSE	The conversation thread is named after the trip. Each trip gets its own agent session.
WHY	Changes like "make day 2 slower" must apply to the right trip, and two trips must never share memory.

DECISION 14

Firestore is the permanent record; the agent's memory is working space

THE QUESTION	Where should the itinerary live?
WHAT I CHOSE	When the agent produces an itinerary, the website saves it to Firestore. Reloading reads Firestore and does not re-plan.
WHY	Agent sessions are for the conversation in progress. The trip belongs to the traveller and should survive anything.

DECISION 15

Every run goes through the one path that carries your identity

THE QUESTION	CopilotKit offers two ways to run the agent. Does it matter which?
WHAT I CHOSE	Always **copilotkit.runAgent**, never the agent's own run method.
WHY	Only the first attaches your Firebase token. The second would send the request without it, and the agent would rightly refuse. I confirmed this in CopilotKit's source before writing the code.

DECISION 16

Failures are shown, not swallowed

THE QUESTION	When a run failed, you saw CopilotKit's developer banner and no proper message. Why?
WHAT I CHOSE	The canvas now listens for the agent's failure events as well as errors, and shows Ije's own message with Try again.
WHY	The agent reports some failures as an event in the stream rather than as an error, so code that only catches errors misses them.

PART 5

Making the agent great

The agent was planning one day, from places it could not verify, with no weather and no way to suggest a stay. You asked to make it great.

DECISION 17

Adopt the agents-cli layout, but keep Ije's own front door

THE QUESTION	You chose agents-cli's project layout for its evaluation tools. It also generated a second web server. Keep it?
WHAT I CHOSE	The agent moved into **agent/agent.py**; **agent/main.py** still serves it and checks identity, unchanged. I removed the generated server, along with unused Terraform and an example file that suggested API keys.
WHY	The generated server offered a way into the agent with no identity check at all, which is exactly what ADR 005 forbids. One front door is easier to guard than two.
WHAT I DID NOT DO	I did not change the AI model. The tooling's own guidance says never to change the model unless asked.

DECISION 18

Real places and weather from Google, with no API keys

THE QUESTION	Where should places, restaurants, hotels and forecasts come from?
WHAT I CHOSE	Google Places API (New) and the Weather API (ADR 006). Both were tested and accept the agent's own identity, so no key exists anywhere.
WHY	Same company and billing as the rest of Ije, so no new party sees user data. It has stable place IDs, opening hours and accessibility details, which a needs-aware planner relies on.
WHAT I DID NOT DO	I did not use OpenStreetMap. It is free, but has no ratings and patchy restaurant and hotel data, which you agreed would make a weaker agent.

DECISION 19

Store place IDs, show names live

THE QUESTION

Google's terms allow storing a place's ID but not its name or address. How do cards show names?

WHAT I CHOSE

Firestore keeps only IDs, Ije's own reasons and links built from the ID. Names are looked up live by **/api/places** each time a card is shown. The agent's reasons are rejected if they repeat a place's name, so names cannot sneak into storage through the AI's own words.

WHY

It keeps Ije inside Google's terms by design, not by luck. Only signed-in users can trigger a lookup, because each one costs money.

THE MOST IMPORTANT IDEA IN THIS PART

The first evaluation showed the agent sometimes ignores an instruction. So the rules that matter most are written into the tools themselves. If the agent breaks one, the tool refuses and tells it exactly what to fix.

DECISION 20

Rules live in the tools, not only in the prompt

THE QUESTION

How do we make sure every plan is complete and real?

WHAT I CHOSE

The save tool rejects a plan unless every day has a morning, afternoon and evening item and at least one place to eat, and every place came from a real search. It also sets each link itself instead of trusting the AI to write one.

WHY

A rejected plan with a precise reason is fixed by the agent within seconds. A plan with a missing day reaching the traveller is a broken promise. These rules are unit-tested (13 tests).

DECISION 21

Needs are added to every search automatically

THE QUESTION

In testing, the agent left a peanut allergy out of its restaurant searches. How do we stop that?

WHAT I CHOSE

The search tool adds the traveller's diet and allergies to every food search, and wheelchair access to every search when it is needed. A fish or shellfish allergy rejects seafood restaurants outright. After saving, the tool tells the agent to remind the traveller to confirm allergens.

WHY

Safety needs should not depend on the AI remembering them. In code, they are applied every time.

DECISION 22

Repair a known AI slip, without ever running code

THE QUESTION	Sometimes Gemini writes its save request as Python inside a tool that does not exist, then repeats the mistake until the run ends with nothing saved, and says it is done. How to handle it?
WHAT I CHOSE	A check after every AI response spots that exact shape and translates it into the real tool call. It reads the text with Python's parser and accepts only plain values; it never runs anything.
WHY	This failure produced the worst outcome possible: a confident claim with nothing behind it. Parsing is safe; running AI-written code would not be.
WHAT I DID NOT DO	I did not give the agent a real code-running tool. It does not need one, and it would be a large new risk.

DECISION 23

Gemini through the global endpoint, with retries

THE QUESTION	During testing Gemini returned "resource exhausted" errors. What should the agent do?
WHAT I CHOSE	The model retries with increasing waits. On your direction, Gemini is called through the global endpoint, now stated explicitly in the deploy (ADR 007).
WHY	The global endpoint routes to wherever there is capacity, which is Google's recommended default. Stored data still stays in us-central1; the governance worksheet records that processing may happen elsewhere.

PART 6

Proving it works

An AI agent cannot be tested like normal code, because its answers vary. So it is evaluated: run on realistic trips and scored.

What is measured

Score	How it works	Why both
plan_rules (0 or 1)	Code reads what the agent actually did: forecast checked first, plan accepted, no unknown tools, nothing changed unasked.	Cannot be talked round by a well-written answer.
ije_judge (1 to 5)	A stronger model (Gemini 2.5 Pro) compares the result with what each case expects, with hard caps for false claims, invented data and ignored needs.	Catches what code cannot: honesty and whether needs were truly respected.

The bar before any deploy: **plan_rules 1 and judge at least 4 on every case**.

The results

Run	plan_rules	judge
First run (baseline)	0.8 average	3.4 average
Final, 5 core trips	1 on every case	4, 4, 5, 5, 5
Final, 3 held-back trips	1 on every case	5, 5, 5

Three real problems the evals caught

- **A plan that did not exist.** The agent called a made-up tool ten times, saved nothing, then said "Expect cloudy weather" as if it had planned. Fixed by the repair in Part 5 and a rule never to claim an unsaved plan.
- **A peanut allergy left out of searches.** Found by a held-back trip. Fixed by adding needs to searches in code.
- **A seafood restaurant for a shellfish allergy.** Fixed by rejecting seafood-type places for those allergies.

DECISION 24

Keep some trips back

THE QUESTION	If you fix the agent against the same test trips, how do you know it is not just learning the test?
WHAT I CHOSE	Two (later three) trips are held back and graded only at the end. When a held-back trip shaped a fix, I added a fresh one, because it was no longer independent.
WHY	This is how the peanut allergy problem was found: by a trip the agent had never been tuned against.

DECISION 25

One judge rule was refined, openly

THE QUESTION	The agent once tried an invented place, the save tool rejected it, and the agent corrected itself. Should that cap the score at 2?
WHAT I CHOSE	Such a caught-and-corrected attempt now costs one point (at most 4). Anything invented that reaches the traveller still caps at 2.
WHY	The traveller saw only real places, because the guardrail worked. Penalising it as if it had reached them would measure the wrong thing.
WHAT I DID NOT DO	I did not lower the bar to make cases pass. The bar stayed at 4 on every case.

DECISION 26

Work around a tooling mismatch, and say so

THE QUESTION	The eval tool seeds each trip in a way the newer agent framework now refuses for security reasons.
WHAT I CHOSE	A small wrapper (**tests/eval/agents_cli_eval.py**) seeds the trip the way the framework accepts, and otherwise runs the eval tool unchanged. It is documented and marked for removal.
WHY	Weakening the framework's security check would have been the wrong fix. Running one trip at a time also avoids rate-limit timeouts.

PART 7

Shipping it

Both the agent and the website are live. Deploying changes things other people rely on, so each step was checked before and after.

DECISION 27

Update the existing agent, check it before trusting it

THE QUESTION How should the new agent replace the old one?

WHAT I CHOSE A dry run first showed it would update the same Agent Runtime engine, keeping its address and identity. After deploying, I confirmed it reports healthy and still refuses requests without a Firebase token.

WHY The website's link to the agent stays the same, and the most important safety property was checked on the live system, not assumed.

DECISION 28

A website image with no secrets in it

THE QUESTION Your local settings file holds the public Firebase config and also private keys. What goes into the website's image?

WHAT I CHOSE The deploy script copies only the public Firebase lines into the build. Next.js and CopilotKit telemetry are switched off. I also removed an old starter script that ran the agent inside the website with an API key.

WHY An image can be pulled and inspected. Only what is safe to publish should ever be in it.

DECISION 29

Public website, without weakening your organisation's security

THE QUESTION Your organisation blocks making Google Cloud services public to everyone. The website must be public.

WHAT I CHOSE Cloud Run's invoker check is turned off for this one service, which is Google's documented approach for this situation. The organisation policy is unchanged; sign-in is still enforced inside the app; the agent behind it stays private.

WHY Overriding an organisation-wide security policy is not my call to make. This keeps the change as small as possible (ADR 005 addendum).

PART 8

Honest notes

What I deliberately did not do, mistakes I made along the way, and what is still open.

Things I chose not to do

- No mock sign-in, demo users or shortcuts, even for testing.
- No test accounts created in your production project. The live signed-in walkthrough is yours.
- No deploys, Google Cloud changes or pushes without your go-ahead.
- No change to the AI model.
- No override of your organisation's security policy.
- No privacy policy before the governance decisions exist.

Mistakes I made, and how they were caught

What happened	How it was caught and fixed
Python cache files slipped into a commit	Spotted in the commit summary; removed before pushing and now ignored.
A local agent session file (which can hold conversation text) was staged	Spotted before committing; removed and ignored.
My first scoring code could not find the user's message in seeded trips	Its scores disagreed with the traces I had read; fixed and re-graded.
A text replacement left a few lines of junk in the search tool	Seen when reading the result; removed before testing.
My test cleanup command stopped its own shell twice	Harmless, and checked each time that your own servers were untouched.

Still open

- **Governance decisions** the privacy policy depends on: controller, lawful basis, the transfer mechanism (now including global processing), retention, and an impact assessment.
- **Leftover keys** in agent/.env (CopilotKit) and frontend/.env (a Google API key). They are unused and not committed, but should be removed and revoked.
- **Not yet proven live:** a signed-in walkthrough on the website, and the AI-slip repair (unit-tested; the slip did not recur in the final run).
- **Known limits:** chat history is not restored after a reload, weather appears only after a run in the current visit, and the agent once repeated a restaurant on two evenings.
- **Before launch:** a budget alert on Maps usage, and rate limiting on the agent route.

Glossary

Term	Plain meaning
ADR	Architecture Decision Record: a short note of a decision, why it was made and what it costs. Ije's are in docs/decisions.
ADK	Google's Agent Development Kit, the framework the Ije agent is built with.
AG-UI	The protocol the website uses to talk to the agent, so the canvas can follow along live.
Agent Runtime	Google's managed service that hosts the Ije agent.
Cloud Run	Google's service that hosts the Ije website.
Eval	A repeatable test of an AI agent: realistic tasks, scored by code and by a judge model.
Firebase ID token	A short-lived, signed proof of who a user is, issued at sign-in and checked by the agent.
Firestore rules	The database's own security rules. They decide who can read and write what, whatever the app does.
Held-back case	An eval case kept out of the fixing loop, so it can show whether fixes really generalise.
OAuth token	A short-lived pass from Google for a named identity. Ije uses these instead of API keys.
Place ID	Google's stable identifier for a place. The only Places data Ije is allowed to store.
Special category data	Data UK law protects more strictly, such as health and religion. Diet and access needs can count.